

CS 550 Operating Systems, Spring 2026
Homework Assignment 4 (HW4)

Out: 3/8/2026 Sun.

Due: 3/15/2026 Sun. 23:59:59

Full points: 100

Note:

- There are 4 questions, each on a separate page.
- Read the submission instructions on the last page carefully and prepare the submission accordingly.

1. (25 points) A producer thread P and a consumer thread C share a single-slot buffer. The programmer intends for:

- the producer to place one item into the buffer via the `put ()` function;
- the consumer to remove that item via the `get ()` function;
- the consumer to wait if the buffer is empty.

The code is shown below.

```
1  pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
2  pthread_cond_t  cv = PTHREAD_COND_INITIALIZER;
3
4  int count = 0;
5  int buffer;
6
7  void put(int x) {
8      buffer = x;
9      count = 1;
10     pthread_cond_signal(&cv);
11 }
12
13 void get() {
14     pthread_mutex_lock(&m);
15     while (count == 0)
16         pthread_cond_wait(&cv, &m);
17     int tmp = buffer;
18     count = 0;
19     printf("got %d\n", tmp);
20     pthread_mutex_unlock(&m);
21 }
```

Assume that there is only one producer thread and only one consumer thread, answer the following questions:

- (1) What problem can the code cause?
- (2) Give an execution scenario that will cause the problem.
- (3) How would you fix the code?

2. (25 points) A multi-threaded producer/consumer program uses the following `consume()` and `produce()` functions. A consumer may consume multiple products at a time, and a producer may also produce multiple products at a time.

```
int products_left = 0;

pthread_cond_t c;
pthread_mutex_t m;

// consume "num" products at a time
void consume(int num) {
    pthread_mutex_lock(&m);
    while (products_left < num)
        pthread_cond_wait(&c, &m);
    ... // consume "num" products
    products_left -= num;
    pthread_mutex_unlock(&m);
}

// produce "num" products at a time
void produce(int num) {
    pthread_mutex_lock(&m);
    ... // produce "num" products
    products_left += num;
    pthread_cond_signal(&c);
    pthread_mutex_unlock(&m);
}
```

After the program has been running for a while, all products are consumed; that is, `products_left == 0`. Then the following three function calls happen close in time, in the following order:

- Thread A calls `consume(10)`
- Thread B calls `consume(100)`
- Thread C calls `produce(50)`

We also know that the buffer can hold up to 1000 products.

Question: What is/are the possible final outcome(s) of this program? For each outcome, describe in one short sentence (i) which thread(s) complete their function call, (ii) which thread(s) remain blocked, and (iii) how many products remain in the buffer at the end.

3. (25 points) A programmer tries to use a semaphore as a lock:

```
#define CNT 3
sem_t s;
int counter = 0;

void* worker(void* arg) {
    sem_wait(&s);
    counter++;
    sem_post(&s);
    return NULL;
}

int main(void) {
    pthread_t thread[CNT];
    sem_init(&s, 0, 2);

    for (int i = 0; i < CNT; i++) {
        pthread_create(&thread[i], NULL, worker, NULL);
    }

    for (int i = 0; i < CNT; i++) {
        pthread_join(&thread[i]);
    }

    printf("%d", counter);
    return 0;
}
```

Assuming all the function calls in the code always succeed, what are the possible outputs of the code?

4. (25 points) The following code uses conditional variable to implement the classic semaphore as we learned in class. Complete the code by filling in blanks (1) to (6).

```
typedef struct {
    int val;
    int available;
    pthread_mutex_t mutex;
    pthread_cond_t cond;
} semaphore;

void sem_init(semaphore *s, int v) {
    // assume that the initial value of the semaphore is always positive
    s->val = v;
    s->available = 1;
}

void sem_down(semaphore *s)
{
    pthread_mutex_lock(&s->mutex);
    s->val ____ 1; // blank (1)
    while (s->available == ____ ) // blank (2)
        pthread_cond_wait(&s->cond, &s->mutex);
    if (s->val ____ 0) // blank (3)
        s->available = ____; // blank (4)
    pthread_mutex_unlock(&s->mutex);
}

void sem_up(semaphore *s) {
    pthread_mutex_lock(&s->mutex);
    s->val ____ 1; // blank (5)
    s->available = ____; // blank (6)
    pthread_cond_signal(&s->cond);
    pthread_mutex_unlock(&s->mutex);
}
```

Question: Fill in blanks (1) to (6).

Submission instructions

1. Type your answers using the text editor you like. Remember to include the index number of each question.
2. Export the file to PDF format.
3. Name the PDF file based on your BU email ID. For example, if your BU email is “abc@binghamton.edu”, then the PDF file should be named as “hw4_abc.pdf”.
4. Failure to follow the above instructions will lead to 5 points off.
5. Submit the PDF file to Brightspace before the deadline.